Category C Problems

M1. Library Inventory Management

Scenario

The library currently tracks its book inventory using three parallel arrays — titles, authors, copiesAvailable — and a recount last week already went out of sync. Rebuild it the OOP way.

Task

- Define a class BookInventory with fields String title, String author, and int copiesAvailable.
- Write a constructor that sets all three fields.
- Write an instance method printEntry() that prints one formatted line.
- Create four BookInventory objects for different books, store them in an array of BookInventory, and print each one in a loop.

Suggested method signature(s)

```
class BookInventory { ... }
void printEntry()
```

Sample Input / Output

Input	Output
Clean Code, Robert C. Martin, 3 *Effective Java, Joshua Bloch, 5* *Refactoring, Martin Fowler, 0* *Design Patterns, GoF, 2*	*Clean Code by Robert C. Martin - 3 copies available* *Effective Java by Joshua Bloch - 5 copies available* *Refactoring by Martin Fowler - 0 copies available* *Design Patterns by GoF - 2 copies available*

M2. Payroll Salary Management

Scenario

A company's payroll account must never let its net salary be set directly from outside the class, and bonuses must never be negative.

Task

- Define a class PayrollAccount with a private double basicSalary and a private double bonus.
- Write a public constructor that accepts an opening basic salary; if a negative value is given, start at 0 instead and print a warning.
- Write a public method creditBonus(double amount) that rejects amount <= 0 with a message, otherwise adds it to bonus.
- Write a public method deductTax(double percent) that reduces basicSalary by that percentage, rejecting any percent outside the 0–100 range with a message.
- Write a public getNetSalary() method that returns basicSalary + bonus for read-only access. There must be no public way to set basicSalary or bonus directly.

Suggested method signature(s)

```
void creditBonus(double amount)
void deductTax(double percent)
double getNetSalary()
```

Sample Input / Output

Input	Output
basicSalary = 50000 *creditBonus(5000)* *deductTax(10)*	*Bonus credited: Rs 5000.0* *Tax deducted: 10%* *Net salary: Rs 50000.0*

M3. Employee Profile Creation

Scenario

Interns join without a fixed salary structure yet; permanent employees join with a known salary from day one. Support both without writing the same setup logic twice.

Task

- Define a class Employee with fields empId, empName, salary, and isIntern.
- Write a constructor Employee(String empId, String empName, double salary) for permanent employees, setting isIntern to false.
- Write a second constructor Employee(String empId, String empName) for interns, which uses this(...) to chain to the three-argument constructor above with salary set to 0, then sets isIntern to true afterwards.
- Write a method printProfile() that prints all four fields on one line.
- Create one Employee object with each constructor and call printProfile() on both.

Suggested method signature(s)

```
public Employee(String empId, String empName, double salary)
public Employee(String empId, String empName)   // chains via this(...)
```

Sample Input / Output

Input	Output
Permanent: E-101, Divya, 65000 *Intern: E-102, Arjun*	*E-101 | Divya | Rs 65000.0 | Intern: false* *E-102 | Arjun | Rs 0.0 | Intern: true*

M4. Exam Hall Ticket Reference Management

Scenario

Two “different” variables in a script both claim to represent Priya's exam hall ticket, but only one of them is actually real. Prove it with code.

Task

- Define a class HallTicket with fields String studentName and int seatNumber, both set through a constructor.
- Create one HallTicket object for Priya. Assign a second variable to point at that same object (do not create a new one).
- Through the second variable, change seatNumber.
- Print the field's value as seen through the first variable, and print whether the two variables are == to each other.
- Create a third, separate HallTicket object with identical field values, and print whether it is == to the first.

Suggested method signature(s)

```
class HallTicket { String studentName; int seatNumber; }
```

Sample Input / Output

Input	Output
HallTicket priya = new HallTicket("Priya", 0); *HallTicket copy = priya;* *copy.seatNumber = 45;* *HallTicket separate = new HallTicket("Priya", 45);*	*Priya's seatNumber (via first variable): 45* *copy == priya: true* *separate == priya: false*

M5. Employee and Company Information Management

Scenario

A trainee developer's first draft of Employee stores empName, salary, AND the company name as instance fields — meaning every single employee object ends up with its own copy of “Bright Horizon Technologies” typed in separately. Fix the design.

Task

- Define a class Employee with instance fields empName and salary, and a static field companyName shared by every employee.
- Add a static field employeeCount that increments once inside the constructor, every time.
- Write a static method printCompanyInfo() that prints companyName and employeeCount — and must not reference any instance field.
- Create three Employee objects, then call printCompanyInfo() through the class name, not through any object.

Suggested method signature(s)

```
static void printCompanyInfo()
```

Sample Input / Output

Input	Output
3 Employee objects created	*Bright Horizon Technologies* *Employees on record: 3*